

第 19 章

绘图及交互

OpenCV 提供了方便的绘图功能，使用其中的绘图函数可以绘制直线、矩形、圆、椭圆等多种几何图形，还能在图像中的指定位置添加文字说明。

在处理图像时，可能需要与当前正在处理的图像进行交互。OpenCV 提供了鼠标事件，用户可以通过鼠标与图像交互。鼠标事件能够识别常用的鼠标操作，例如：针对不同按键的单击、双击，鼠标的滑动、拖曳等。

OpenCV 还提供了滚动条用于实现交互功能。用户可以拖动滚动条在某一个范围内设置特定的值，并将该值应用于后续的图像处理中。而且，如果设置为二值形式，滚动条还可以作为开关选择器使用。

绘图、鼠标交互、滚动条交互都是 OpenCV 中 GUI 的重要知识，本章将对上述内容做简单介绍。

19.1 绘画基础

OpenCV 提供了绘制直线的函数 `cv2.line()`、绘制矩形的函数 `cv2.rectangle()`、绘制圆的函数 `cv2.circle()`、绘制椭圆的函数 `cv2.ellipse()`、绘制多边形的函数 `cv2.polylines()`、在图像内添加文字的函数 `cv2.putText()` 等多种绘图函数。

这些绘图函数有一些共有的参数，主要用于设置源图像、颜色、线条属性等。下面对这些共有参数做简单的介绍。

- **img**: 在其上面绘制图形的载体图像（绘图的容器载体，也称为画布、画板）。
- **color**: 绘制形状的颜色。通常使用 BGR 模型表示颜色，例如，(0, 255, 0)表示绿色。对于灰度图像，只能传入灰度值。需要注意，颜色通道的顺序是 BGR，而不是 RGB。
- **thickness**: 线条的粗细。默认值是 1，如果设置为-1，表示填充图形（即绘制的图形是实心的）。
- **lineType**: 线条的类型，默认是 8 连接类型。`lineType` 参数的值及说明如表 19-1 所示。

表 19-1 `lineType` 参数的值及说明

参数	说明
<code>cv2.FILLED</code>	填充
<code>cv2.LINE_4</code>	4 连接类型
<code>cv2.LINE_8</code>	8 连接类型
<code>cv2.LINE_AA</code>	抗锯齿，该参数会让线条更平滑

- **shift**: 数据精度。该参数用来控制数值（例如圆心坐标等）的精度，一般情况下不需要设置。

19.1.1 绘制直线

OpenCV 提供了函数 `cv2.line()` 用来绘制直线（线段）。该函数的语法格式为：

```
img = cv2.line( img, pt1, pt2, color[, thickness[, lineType ]])
```

式中：

- 参数 `img`、`color`、`thickness`、`lineType` 的含义如前面的说明所示。
- `pt1` 表示线段的第 1 个点（起点）。
- `pt2` 表示线段的第 2 个点（终点）。

【例 19.1】 使用 `cv2.line()` 函数在一个黑色背景图像内绘制三条线段。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
n = 300
img = np.zeros((n+1,n+1,3), np.uint8)
img = cv2.line(img, (0,0), (n,n), (255,0,0), 3)
img = cv2.line(img, (0,100), (n,100), (0,255,0), 1)
img = cv2.line(img, (100,0), (100,n), (0,0,255), 6)
winname = 'Demo19.1'
cv2.namedWindow(winname)
cv2.imshow(winname, img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

运行程序，结果如图 19-1 所示。该程序在图像 `img` 中使用函数 `cv2.line()` 绘制了三条不同起始点、颜色和粗细的直线。

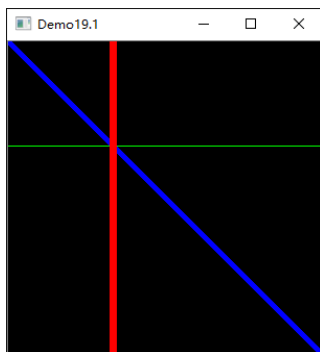


图 19-1 绘制直线示例

19.1.2 绘制矩形

OpenCV 提供了函数 `cv2.rectangle()` 用来绘制矩形。该函数的语法格式为：

```
img = cv2.rectangle( img, pt1, pt2, color[, thickness[, lineType]] )
```

式中:

- 参数 `img`、`color`、`thickness`、`lineType` 的含义如前面的说明所示。
- `pt1` 为矩形顶点。
- `pt2` 为矩形中与 `pt1` 对角的顶点。

【例 19.2】 使用函数 `cv2.rectangle()` 在一个白色背景图像内绘制一个实心矩形。

根据题目的要求, 编写代码如下:

```
import numpy as np
import cv2
n = 300
img = np.ones((n,n,3), np.uint8)*255
img = cv2.rectangle(img, (50,50), (n-100,n-50), (0,0,255), -1)
winname = 'Demo19.1'
cv2.namedWindow(winname)
cv2.imshow(winname, img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

运行程序, 结果如图 19-2 所示。该段程序在图像 `img` 中使用函数 `cv2.rectangle()` 绘制了一个红色的实心矩形 (在纸质书中显示为灰色)。

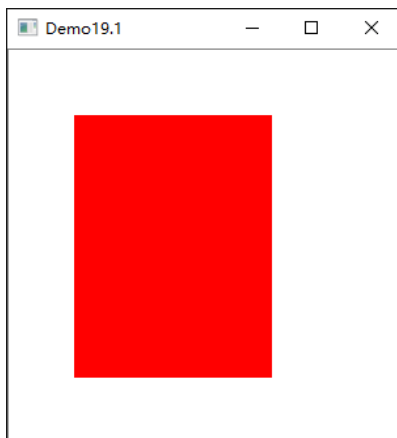


图 19-2 绘制矩形示例

19.1.3 绘制圆形

OpenCV 提供了函数 `cv2.circle()` 用来绘制圆。该函数的语法格式为:

```
img = cv2.circle( img, center, radius, color[, thickness[, lineType]] )
```

式中:

- 参数 `img`、`color`、`thickness`、`lineType` 的含义如前面的说明所示。
- `center` 为圆心。



- radius 为半径。

【例 19.3】 使用函数 `cv2.circle()` 在一个白色背景图像内绘制一组同心圆。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
d = 400
img = np.ones((d,d,3),dtype="uint8")*255
(centerX,centerY) = (round(img.shape[1] / 2),round(img.shape[0] / 2))
# 将图像的中心作为圆心,实际值为 d/2
red = (0,0,255) # 设置白色变量
for r in range(5,round(d/2),12):
    cv2.circle(img,(centerX,centerY),r,red,3)
    # circle(载体图像, 圆心, 半径, 颜色)
cv2.imshow("Demo19.3",img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

运行程序，结果如图 19-3 所示。该段程序在图像 `img` 中使用函数 `cv2.circle()` 绘制了一组同心圆。

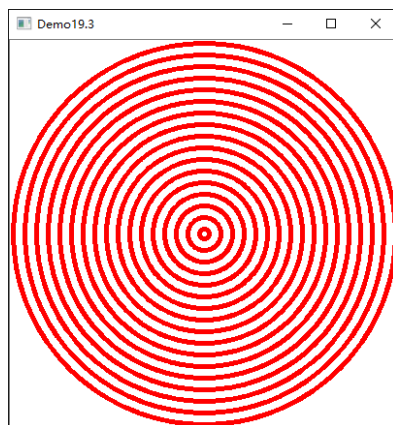


图 19-3 绘制同心圆示例

【例 19.4】 使用函数 `cv2.circle()` 在一个白色背景图像内绘制一组位置和大小均随机的实心圆。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
d = 400
img = np.ones((d,d,3),dtype="uint8")*255
# 生成白色背景
for i in range(0,100):
    centerX = np.random.randint(0,high = d)
    # 生成随机圆心 centerX, 确保在画布 img 内
```

```

centerY = np.random.randint(0,high = d)
# 生成随机圆心 centerY, 确保在画布 img 内
radius = np.random.randint(5,high = d/5)
# 生成随机半径, 值范围为[5,d/5), 最大半径是 d/5
color = np.random.randint(0,high = 256,size = (3,)).tolist()
# 生成随机颜色, 3 个[0,256)的随机数
cv2.circle(img,(centerX,centerY),radius,color,-1)
# 使用上述随机数在画布 img 内画圆
cv2.imshow("demo19.4",img)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

运行程序, 结果如图 19-4 所示。该段程序在画布中使用函数 `cv2.circle()` 绘制了一组随机的实心圆。

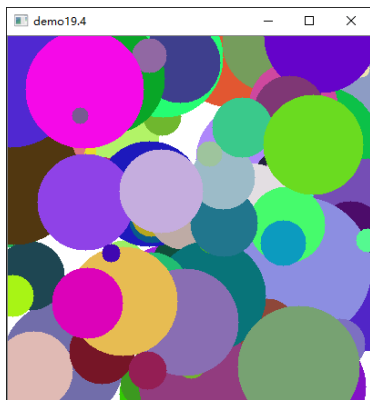


图 19-4 绘制实心圆示例

19.1.4 绘制椭圆

OpenCV 提供了函数 `cv2.ellipse()` 用来绘制椭圆。该函数的语法格式为:

```

img=cv2.ellipse(img, center, axes, angle, startAngle, endAngle, color[,
thickness[, lineType]])

```

式中:

- 参数 `img`、`color`、`thickness`、`lineType` 的含义如前面的说明所示。
- `center` 为椭圆的圆心坐标。
- `axes` 为轴的长度。
- `angle` 为偏转的角度。
- `startAngle` 为圆弧起始角的角度。
- `endAngle` 为圆弧终结角的角度。

为了方便理解, OpenCV 官网上对上述参数的含义做了配图说明, 有兴趣的读者可以去官网进一步了解。

【例 19.5】 使用函数 `cv2.ellipse()` 在一个白色背景图像内随机绘制一组空心椭圆。



根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
d = 400
img = np.ones((d,d,3),dtype="uint8")*255
# 生成白色背景
center=(round(d/2),round(d/2))
# 注意数值类型，不可以使用语句 center=(d/2,d/2)
size=(100,200)
# 轴的长度
for i in range(0,10):
    angle = np.random.randint(0,361)
    # 偏移角度
    color = np.random.randint(0,high = 256,size = (3,)).tolist()
    # 生成随机颜色，3 个[0,256)的随机数
    thickness = np.random.randint(1,9)
    cv2.ellipse(img, center, size, angle, 0, 360, color,thickness)
cv2.imshow("demo19.5",img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

运行程序，结果如图 19-5 所示。该段程序在图像 `img` 中使用函数 `cv2.ellipse()` 绘制了一组随机的空心椭圆。

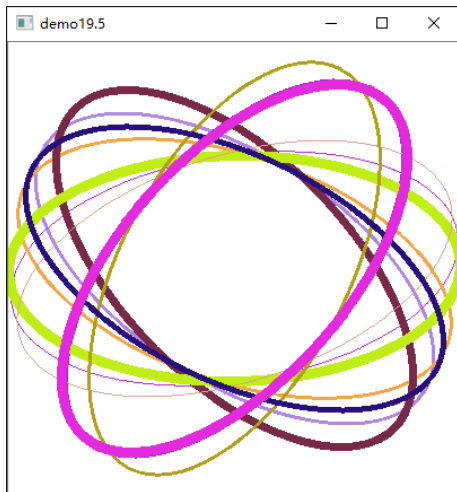


图 19-5 绘制空心椭圆示例

19.1.5 绘制多边形

OpenCV 提供了函数 `cv2.polylines()` 用来绘制多边形。该函数的语法格式为：

```
img = cv2.polylines( img, pts, isClosed, color[, thickness[, lineType[, shift]]])
```

式中：

- 参数 `img`、`color`、`thickness`、`lineType` 和 `shift` 的含义如前面的说明所示。
- `pts` 为多边形的各个顶点。
- `isClosed` 为闭合标记，用来指示多边形是否是封闭的。若该值为 `True`，则将最后一个点与第一个点连接，让多边形闭合；否则，仅仅将各个点依次连接起来，构成一条曲线。

在使用函数 `cv2.polylines()` 绘制多边形时，需要给出每个顶点的坐标。这些点的坐标构建了一个大小等于“顶点个数*1*2”的数组，这个数组的数据类型必须为 `numpy.int32`。下面的例 19.6 绘制了一个黄色的具有四个顶点的多边形。

【例 19.6】 使用函数 `cv2.polylines()` 在一个白色背景图像内绘制一个多边形。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
d = 400
img = np.ones((d,d,3),dtype="uint8")*255
# 生成白色背景
pts=np.array([[200,50],[300,200],[200,350],[100,200]], np.int32)
# 生成各个顶点,注意数据类型为 int32
pts=pts.reshape((-1,1,2))
# 第 1 个参数为-1, 表明它未设置具体值, 它所表示的维度值是通过其他参数值计算得到的
cv2.polylines(img,[pts],True,(0,255,0),8)
# 调用函数 cv2.polylines() 完成多边形绘图。注意, 第 3 个参数控制多边形是否封闭
cv2.imshow("demo19.6",img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

运行程序，结果如图 19-6 所示。该段程序在图像 `img` 中使用函数 `cv2.polylines()` 绘制了一个封闭的多边形。

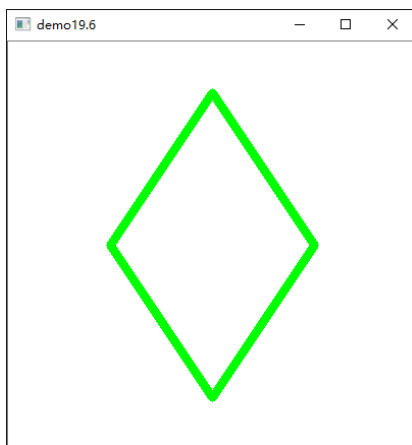


图 19-6 绘制封闭的多边形示例

函数 `cv2.polylines()` 中的第 3 个参数 `isClosed` 是闭合标记，将该值设置为 `False` 时，仅仅将各个顶点用线段连接，多边形是不封闭的。此时的代码为：

```
cv2.polylines(img,[pts],False,(0,255,0),8)
```



上述代码绘制的结果如图 19-7 所示。

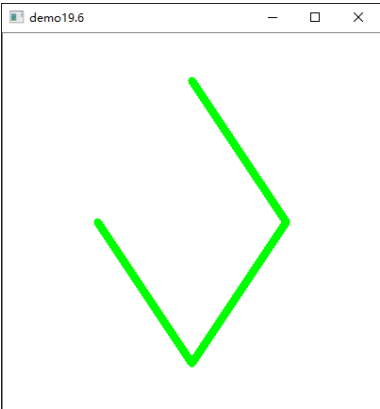


图 19-7 不封闭的多边形示例

因此，可以使用函数 `cv2.polylines()` 来绘制多条首尾相连的线段。只要把线段的各个点放在一个数组中，将这个数组传给函数 `cv2.polylines()` 的第 2 个参数 `pts` 就可以了。

19.1.6 在图形上绘制文字

OpenCV 提供了函数 `cv2.putText()` 用来在图形上绘制文字。该函数的语法格式为：

```
img=cv2.putText(img, text, org, fontFace, fontScale, color[, thickness[,  
lineType[, bottomLeftOrigin]]])
```

式中：

- 参数 `img`、`color`、`thickness`、`lineType` 和 `shift` 的含义如前面的说明所示。
- `text` 为要绘制的字体。
- `org` 为绘制字体的位置，以文字的左下角为起点。
- `fontFace` 表示字体类型，其参数类型及含义如表 19-2 所示。
- `fontScale` 表示字体大小。
- `bottomLeftOrigin` 用于控制文字的方向。默认值为 `False`，当设置为 `True` 时，文字是垂直镜像的效果。

表 19-2 参数 `fontFace` 的值及含义

值	含义
<code>cv2.FONT_HERSHEY_SIMPLEX</code>	正常大小的 sans-serif 字体
<code>cv2.FONT_HERSHEY_PLAIN</code>	小号的 sans-serif 字体
<code>cv2.FONT_HERSHEY_DUPLEX</code>	正常大小的 sans-serif 字体（比 <code>cv2.FONT_HERSHEY_SIMPLEX</code> 更复杂）
<code>cv2.FONT_HERSHEY_COMPLEX</code>	正常大小的 serif 字体
<code>cv2.FONT_HERSHEY_TRIPLEX</code>	正常大小的 serif 字体（比 <code>cv2.FONT_HERSHEY_COMPLEX</code> 更复杂）
<code>cv2.FONT_HERSHEY_COMPLEX_SMALL</code>	<code>cv2.FONT_HERSHEY_COMPLEX</code> 字体的简化版
<code>cv2.FONT_HERSHEY_SCRIPT_SIMPLEX</code>	手写风格的字体
<code>cv2.FONT_HERSHEY_SCRIPT_COMPLEX</code>	<code>cv2.FONT_HERSHEY_SCRIPT_SIMPLEX</code> 字体的进阶版
<code>cv2.FONT_ITALIC</code>	斜体标记

【例 19.7】使用函数 `cv2.putText()` 在一个白色背景图像内绘制文字。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
d = 400
img = np.ones((d,d,3),dtype="uint8")*255
# 生成白色背景
font=cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(img, 'OpenCV', (0,200), font, 3, (0,255,0),15)
cv2.putText(img, 'OpenCV', (0,200), font, 3, (0,0,255),5)
cv2.imshow("demo19.7",img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

运行程序，结果如图 19-8 所示。该段程序在图像 `img` 中使用函数 `cv2.putText()` 绘制了文字“OpenCV”。

在上述程序中，第 1 次调用函数 `cv2.putText()` 绘制了一个宽度（由参数 `thickness` 控制）为 15 的文字“OpenCV”，第 2 次调用函数 `cv2.putText()` 时，在第 1 次绘制的“OpenCV”内部绘制了一个稍细的宽度为 5 的“OpenCV”。因为两次使用的颜色不一样，所以实现了文字的“描边”效果。

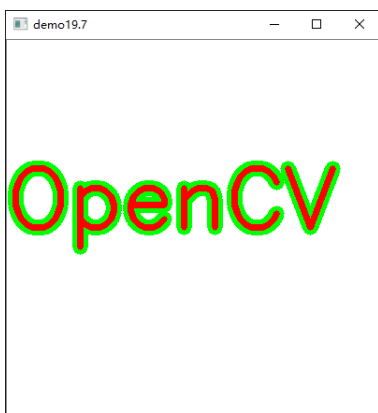


图 19-8 绘制文字示例

【例 19.8】使用函数 `cv2.putText()` 在一个白色背景图像内绘制一段镜像的文字。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
d = 400
img = np.ones((d,d,3),dtype="uint8")*255
# 生成白色背景
font=cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(img, 'OpenCV', (0,150), font, 3, (0,0,255),15)
cv2.putText(img, 'OpenCV', (0,250), font, 3, (0,255,0),15,
```



```
cv2.FONT_HERSHEY_SCRIPT_SIMPLEX, True)
cv2.imshow("demo19.7",img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

运行程序，结果如图 19-9 所示。该段程序在图像 img 中使用函数 cv2.putText()绘制了两种不同形式的文字“OpenCV”。

在上述程序中，第 1 次调用函数 cv2.putText()绘制了上方的正常显示的文字“OpenCV”；第 2 次调用函数 cv2.putText()时，参数 bottomLeftOrigin 的值被设置为 True，实现绘制镜像文字效果，绘制了下方的倒置文字“OpenCV”。

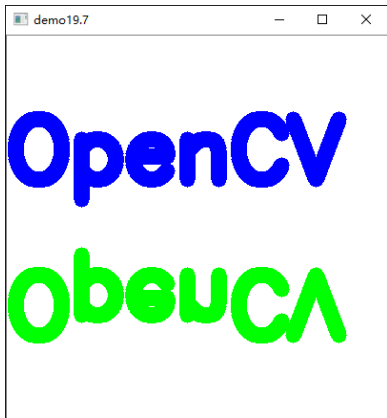


图 19-9 绘制镜像文字示例

19.2 鼠标交互

当用户触发鼠标事件时，我们希望对该事件做出响应。例如，用户单击鼠标，我们就画一个圆。通常的做法是，创建一个 OnMouseAction()响应函数，将要实现的操作写在该响应函数内。响应函数是按照固定的格式创建的，其格式为：

```
def OnMouseAction(event,x,y,flags,param):
```

式中：

- event 表示触发了何种事件，具体事件如表 19-3 所示。
- x, y 代表触发鼠标事件时，鼠标在窗口中的坐标(x, y)。
- flags 代表鼠标的拖曳事件，以及键盘鼠标联合事件，如表 19-4 所示。
- param 为函数 ID，标识所响应的事件函数，相当于自定义一个 OnMouseAction()函数的 ID。
- OnMouseAction 为响应函数的名称，该名称可以自定义。

表 19-3 参数 event 的值以及含义

值	含义
cv2.EVENT_LBUTTONDOWNCLK	双击左键
cv2.EVENT_LBUTTONDOWN	按下左键

续表

值	含义
cv2.EVENT_LBUTTONDOWN	抬起左键
cv2.EVENT_MBUTTONDOWNBLCLK	双击中间键
cv2.EVENT_MBUTTONDOWNDOWN	按下中间键
cv2.EVENT_MBUTTONDOWNUP	抬起中间键
cv2.EVENT_MOUSEHWHEEL	滚轮滑动（正值和负值分别表示向左和向右滚动）
cv2.EVENT_MOUSEMOVE	鼠标滑动
cv2.EVENT_MOUSEWHEEL	滚轮滑动（正值和负值分别表示向前和向后滚动）
cv2.EVENT_RBUTTONDOWNBLCLK	双击右键
cv2.EVENT_RBUTTONDOWNDOWN	按下右键
cv2.EVENT_RBUTTONDOWNUP	抬起右键

表 19-4 参数 flags 的值及含义

值	含义
cv2.EVENT_FLAG_ALTKEY	按下 Alt 键
cv2.EVENT_FLAG_CTRLKEY	按下 Ctrl 键
cv2.EVENT_FLAG_LBUTTON	左键拖曳
cv2.EVENT_FLAG_MBUTTON	中间键拖曳
cv2.EVENT_FLAG_RBUTTON	右键拖曳
cv2.EVENT_FLAG_SHIFTKEY	按下 Shift 键

定义响应函数以后，要将该函数与一个特定的窗口建立联系（绑定），让该窗口内的鼠标触发事件时，能够找到该响应函数并执行。要将函数与窗口绑定，可以通过函数 cv2.setMouseCallback()实现，其基本语法格式是：

```
cv2.setMouseCallback(winname,onMouse)
```

式中：

- winname 为绑定的窗口名。
- onMouse 为绑定的响应函数名。

【例 19.9】设计一个程序，对触发的鼠标事件进行判断。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
def Demo(event,x,y,flags,param):
    if event == cv2.EVENT_LBUTTONDOWN:
        print("单击了鼠标左键")
    elif event==cv2.EVENT_RBUTTONDOWN :
        print("单击了鼠标右键")
    elif flags==cv2.EVENT_FLAG_LBUTTON:
        print("按住左键拖动了鼠标")
    elif event==cv2.EVENT_MBUTTONDOWN :
        print("单击了中间键")
# 创建名称为 Demo 的响应（回调）函数 OnMouseAction
```



```
# 将响应函数 Demo 与窗口“Demo19.9”建立连接（实现绑定）
img = np.ones((300,300,3),np.uint8)*255
cv2.namedWindow('Demo19.9')
cv2.setMouseCallback('Demo19.9',Demo)
cv2.imshow('Demo19.9',img)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，在创建的窗口 Demo19.9 内：

- 单击鼠标左键，会触发单击左键事件 “cv2.EVENT_LBUTTONDOWN”。
- 单击鼠标右键，会触发单击右键事件 “cv2.EVENT_RBUTTONDOWN”。
- 单击鼠标中间键（滚轮），会触发单击中间键事件 “cv2.EVENT_MBUTTONDOWN”
- 按住鼠标左键拖动鼠标，会依次触发单击左键事件 “cv2.EVENT_LBUTTONDOWN” 和左键拖动事件 “cv2.EVENT_FLAG_LBUTTON”。

运行上述程序，先后单击左键、右键、滚轮、拖动鼠标，则程序运行结果如图 19-10 所示。

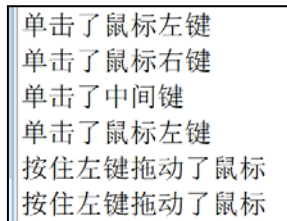


图 19-10 【例 19.9】程序的运行结果

说明：可以通过下面的方法查看 OpenCV 所支持的鼠标事件：

```
import cv2
events=[i for i in dir(cv2) if 'EVENT' in i]
print(events)
```

19.2.1 简单示例

本节实现一个双击鼠标绘制矩形的简单程序。在该程序中，首先创建响应函数 draw()，在该函数内编写代码，实现：当双击鼠标时，以当前位置为顶点绘制大小随机、颜色随机的矩形。通过函数 cv2.setMouseCallback()将函数 draw()和新建窗口“Demo19.10”绑定。

【例 19.10】设计一个程序，当双击鼠标后，以当前位置为顶点绘制大小随机、颜色随机的矩形。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
d = 400
def draw(event,x,y,flags,param):
    if event==cv2.EVENT_LBUTTONDBLCLK:
        p1x=x
```

```

p1y=y
p2x=np.random.randint(1,d-50)
p2y=np.random.randint(1,d-50)
color = np.random.randint(0,high = 256,size = (3,)).tolist()
cv2.rectangle(img, (p1x,p1y), (p2x,p2y),color,2)
img = np.ones((d,d,3),dtype="uint8")*255
cv2.namedWindow('Demo19.10')
cv2.setMouseCallback('Demo19.10',draw)
while(1):
    cv2.imshow('Demo19.10',img)
    if cv2.waitKey(20)==27:
        break
cv2.destroyAllWindows()

```

运行程序，在图像内双击鼠标就会以当前位置为顶点绘制大小随机、颜色随机的矩形。图 19-11 是在图像的不同位置多次双击鼠标后的结果。

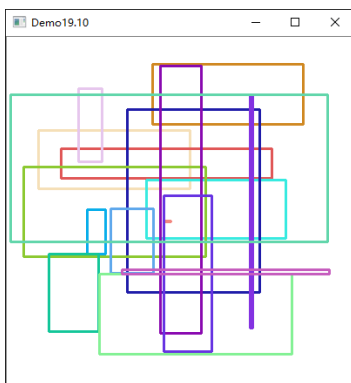


图 19-11 双击鼠标绘制圆形

19.2.2 进阶示例

鼠标事件为用户与计算机之间的交互提供了接口，让用户能够非常灵活地与计算机之间实现交互。本节通过一个鼠标与键盘配合的例子，展示如何实现用户与计算机的交互。

【例 19.11】设计一个交互程序，通过键盘与鼠标的组合控制显示不同的形状或文字。

根据题目的要求，编写代码如下：

```

import cv2
import numpy as np
thickness=-1
mode=1
d=400
def draw_circle(event,x,y,flags,param):
    if event==cv2.EVENT_LBUTTONDOWN:
        a=np.random.randint(1,d-50)
        r=np.random.randint(1,d/5)
        angle = np.random.randint(0,361)

```



```

color = np.random.randint(0,high = 256,size = (3,)).tolist()
if mode==1:
    cv2.rectangle(img,(x,y),(a,a),color,thickness)
elif mode==2:
    cv2.circle(img,(x,y),r,color,thickness)
elif mode==3:
    cv2.line(img,(a,a),(x,y),color,3)
elif mode==4:
    cv2.ellipse(img,(x,y),(100,150),angle,0,360,color,thickness)
elif mode==5:
    cv2.putText(img,'OpenCV',(0,round(d/2)),
                cv2.FONT_HERSHEY_SIMPLEX,2,color,5)
img=np.ones((d,d,3),np.uint8)*255
cv2.namedWindow('image')
cv2.setMouseCallback('image',draw_circle)
while(1):
    cv2.imshow('image',img)
    k=cv2.waitKey(1)&0xFF
    if k==ord('r'):
        mode=1
    elif k==ord('c'):
        mode=2
    elif k==ord('l'):
        mode=3
    elif k==ord('e'):
        mode=4
    elif k==ord('t'):
        mode=5
    elif k==ord('f'):
        thickness=-1
    elif k==ord('u'):
        thickness=3
    elif k==27:
        break
cv2.destroyAllWindows()

```

运行程序，我们就可以通过键盘控制单击鼠标左键时在图像内显示的对象。不同的按键，控制不同的显示内容：

- 按下“r”键后，在窗体¹内单击鼠标左键，会显示以当前鼠标位置为顶点，以任意位置为对角顶点的一个矩形。
- 按下“c”键后，在窗体内单击鼠标左键，会显示以当前鼠标位置为圆心，以随机数为半径的一个圆。
- 按下“l”键后，在窗体内单击鼠标左键，会显示以当前鼠标位置为端点，以任意位置

1 为了强调了交互性，使用了“窗体”，与前面所说的“窗口”含义并无太大区别。

为另一个端点的线段。

- 按下“e”键后，在窗体内单击鼠标左键，会显示以当前鼠标位置为中心点，角度随机的一个椭圆。
- 按下“t”键后，在窗体内单击鼠标左键，会在当前窗体内显示颜色随机的文字“OpenCV”。

此外，还通过“f”键和“u”键控制各种图形是否为实心：

- 按下“f”键后，图形为实心。
- 按下“u”键后，图形为空心。

如果要结束程序，可以按“Esc”键（ASCII 码为 27），此时程序退出循环，停止运行。

运行程序，按照上述控制方式绘制不同的对象，可以得到如图 19-12 所示的运行结果。

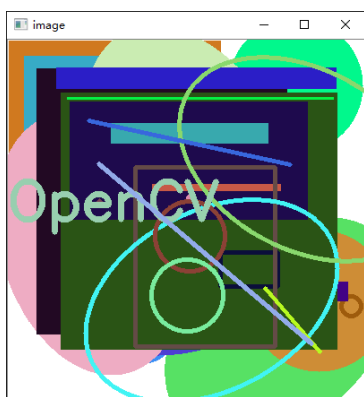


图 19-12 【例 19.11】程序的运行结果

19.3 滚动条

滚动条（Trackbar）在 OpenCV 中是非常方便的交互工具，它依附于特定的窗口而存在。通过调节滚动条能够设置、获取指定范围内的特定值。

在 OpenCV 中，函数 `cv2.createTrackbar()` 用来定义滚动条，其语法格式为：

```
cv2.createTrackbar(trackbarname, winname, value, count, onChange)
```

式中：

- `trackbarname` 为滚动条的名称。
- `winname` 为滚动条所依附窗口的名称。
- `value` 为初始值，该值决定滚动条中滑块的位置。
- `count` 为滚动条的最大值。通常情况下，其最小值是 0。
- `onChange` 为回调函数。一般情况下，将滚动条改变后要实现的操作写在回调函数内。

函数 `cv2.createTrackbar()` 用于生成一个滚动条。拖动滚动条，就可以设置滚动条的值，并让滚动条返回对应的值。滚动条的值可以通过函数 `cv2.getTrackbarPos()` 获取，其语法格式为：

```
retval=getTrackbarPos( trackbarname, winname )
```



式中：

- `retval` 为返回值，获取函数 `cv2.createTrackbar()` 生成的滚动条的值。
- `trackbarname` 为滚动条的名称。
- `winname` 为滚动条所依附的窗口的名称。

19.3.1 用滚动条实现调色板

在 RGB 颜色空间中，任何颜色都是由红（R）、绿（G）、蓝（B）三种颜色构成的，每一种颜色分量的区间是[0, 255]。本节用函数 `cv2.createTrackbar()` 和函数 `cv2.getTrackbarPos()` 设计一个模拟调色板：在窗体中，有三个滚动条分别用来设置 R、G、B 的值，调色板会根据当前的 R、G、B 值实时显示其所对应的颜色。

【例 19.12】 设计一个滚动条交互程序，通过滚动条模拟调色板效果。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
def changeColor(x):
    r=cv2.getTrackbarPos('R','image')
    g=cv2.getTrackbarPos('G','image')
    b=cv2.getTrackbarPos('B','image')
    img[:]=[b,g,r]
img=np.zeros((100,700,3),np.uint8)
cv2.namedWindow('image')
cv2.createTrackbar('R','image',0,255,changeColor)
cv2.createTrackbar('G','image',0,255,changeColor)
cv2.createTrackbar('B','image',0,255,changeColor)
while(1):
    cv2.imshow('image',img)
    k=cv2.waitKey(1)&0xFF
    if k==27:
        break
cv2.destroyAllWindows()
```

运行程序，在窗体对象内，分别调整 R、G、B 三个滚动条，可以得到运行结果如图 19-13 所示。

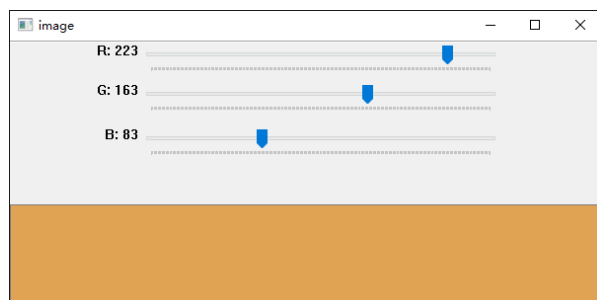


图 19-13 模拟调色板

19.3.2 用滚动条控制阈值处理参数

本节设计一个程序，让不同的滚动条对应不同的响应函数，使滚动条具有更强的交互性。

【例 19.13】设计一个滚动条交互程序，通过滚动条控制函数 `cv2.threshold()` 中的阈值和模式。

首先，回顾一下函数 `cv2.threshold()` 的语法格式：

```
retval, dst=cv2.threshold(src, thresh, maxval, type)
```

式中各参数的含义为：

- `src`：源图像。
- `thresh`：阈值。
- `maxval`：当 `type` 参数的值为 `THRESH_BINARY` 或者 `THRESH_BINARY_INV` 时，需要设定的最大值。
- `type`：阈值处理的方式（类型）。
- `dst`：处理结果。
- `retval`：返回的阈值。

根据题目的要求，编写代码如下：

```
import cv2
Type=0 # 阈值处理方式
Value=0 # 使用的阈值
def onType(a):
    Type= cv2.getTrackbarPos(tType, windowName)
    Value= cv2.getTrackbarPos(tValue, windowName)
    ret, dst = cv2.threshold(o, Value, 255, Type)
    cv2.imshow(windowName, dst)

def onValue(a):
    Type= cv2.getTrackbarPos(tType, windowName)
    Value= cv2.getTrackbarPos(tValue, windowName)
    ret, dst = cv2.threshold(o, Value, 255, Type)
    cv2.imshow(windowName, dst)

o = cv2.imread("lena512.bmp", 0)
windowName = "Demo19.13" # 窗体名
cv2.namedWindow(windowName)
cv2.imshow(windowName, o)
# 创建两个滚动条
tType = "Type" # 用来选取阈值处理方式的滚动条
tValue = "Value" # 用来选取阈值的滚动条
cv2.createTrackbar(tType, windowName, 0, 4, onType)
cv2.createTrackbar(tValue, windowName, 0, 255, onValue)
if cv2.waitKey(0) == 27:
    cv2.destroyAllWindows()
```



运行程序，在窗体对象内，同时显示控制阈值和阈值处理方式的两个滚动条。调整滚动条可以分别控制阈值处理时所使用的阈值和阈值处理方式，运行结果如图 19-14 所示。

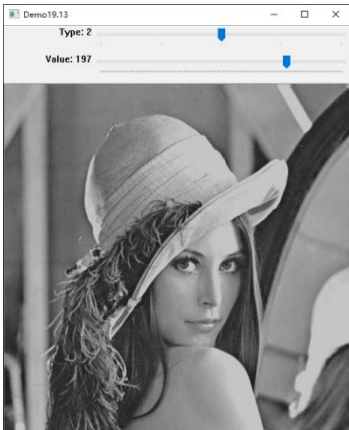


图 19-14 用滚动条控制阈值处理参数

19.3.3 用滚动条作为开关

有时也可将滚动条作为“开关”使用。此时，滚动条只有两种值“0”和“1”，当滚动条的值为 0 时，代表 False；当滚动条的值为 1 时，代表 True。

除了表示逻辑关系外，也可以让滚动条的 0 和 1 分别表示其他任意形式的两种不同状态。

【例 19.14】设计一个滚动条交互程序，用滚动条控制绘制的矩形是实心的还是空心的。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
d=400
global thickness
thickness=-1
def fill(x):
    pass
def draw(event,x,y,flags,param):
    if event==cv2.EVENT_LBUTTONDOWN:
        p1x=x
        p1y=y
        p2x=np.random.randint(1,d-50)
        p2y=np.random.randint(1,d-50)
        color = np.random.randint(0,high = 256,size = (3,)).tolist()
        cv2.rectangle(img, (p1x,p1y), (p2x,p2y),color,thickness)

img=np.ones((d,d,3),np.uint8)*255
cv2.namedWindow('image')
cv2.setMouseCallback('image',draw)
cv2.createTrackbar('R','image',0,1,fill)
```

```

while(1):
    cv2.imshow('image',img)
    k=cv2.waitKey(1)&0xFF
    g=cv2.getTrackbarPos('R','image')
    if g==0:
        thickness=-1
    else:
        thickness=2
    if k==27:
        break
cv2.destroyAllWindows()

```

运行程序，调整滚动条，绘制不同形式的矩形，得到如图 19-15 所示的结果。

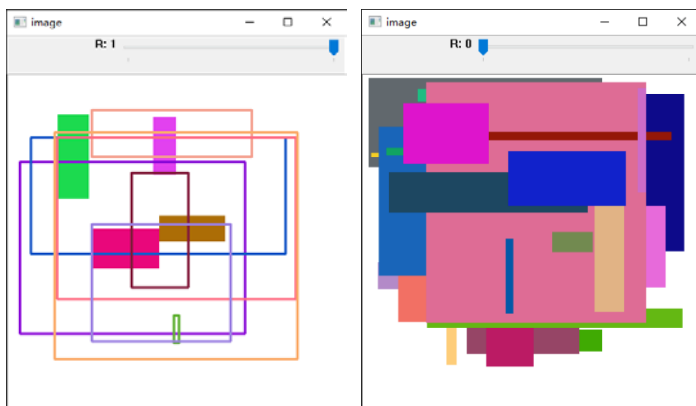


图 19-15 用滚动条控制绘制的矩形是空心的还是实心的